UNIVERSIDAD TECNOLÓGICA NACIONAL
Tecnicatura Universitaria en Programación

TRABAJO PRÁCTICO - UNIDAD 8

Food Store en la feria
La base de la tablet y el cierre de la capa de datos

Proyecto integrador: Food Store
SQLite 3.54 · PostgreSQL 18.6 · Sincronización simulada

Materia: Base de Datos II
Carrera: Tecnicatura Universitaria en Programación (UTN)

Autores:
Lemus Nahuel
Anglat Juan
Elias Juan Cruz

Año 2026

1. La base de la tablet

La tablet no es un cliente de PostgreSQL: es una foto del catálogo y una cola de propuestas. PostgreSQL sigue siendo la fuente de verdad de precio, stock y ventas. SQLite guarda solo lo mínimo para vender sin señal y para explicar cada conflicto.

1.a Qué datos viajan

Viaja a la tablet	No viaja	Por qué
id, nombre	cliente, mail, celular	Identifica el producto; no lleva datos personales.
precio_centavos (entero)	precio decimal interno	Comparaciones exactas, sin redondeo flotante.
stock_visto	stock autoritativo	Es una foto; la tablet no reserva ni descuenta.
disponible, eliminado	ventas/pedidos confirmados	La baja lógica se representa con lápida; la venta la decide el servidor.
fecha_copia	usuarios, roles, hashes	Detecta una foto vieja sin exponer seguridad.

Fuente de verdad: PostgreSQL decide precio, stock y venta. MongoDB (U7) aporta la ficha ampliada del producto, pero no decide precio ni stock. La tablet solo propone.

1.b Las tablas (SQLite STRICT)

Tabla	Claves y reglas
producto_cache	PK id; precio_centavos >= 0; stock_visto >= 0; disponible y eliminado en 0/1; fecha_copia de 10 caracteres.
pedido_borrador	PK id_local UUID de 36; forma_pago solo EFECTIVO, TARJETA o TRANSFERENCIA; fecha ISO.
item_borrador	PK (id_local, producto_id); FK a borrador y a caché; cantidad > 0; precio visto no negativo.
cola_sync	PK solicitud_uuid; FK al borrador; estado PENDIENTE, CONFLICTO, RECHAZADO o SIMULADO_APROBADO.

El script empieza con PRAGMA foreign_keys = ON; y usa CHECK para que la base rechace estados, precios y cantidades inválidas. Además se crean snapshot_servidor (CSV crudo) y vw_catalogo_disponible (C1).

1.c Pruebas: borrador, ROLLBACK, C1 y C7

```
-- Borrador completo
BEGIN;
INSERT INTO pedido_borrador ...;
INSERT INTO item_borrador ...;
INSERT INTO cola_sync ...;
COMMIT;
-- Resultado: borrador|1  items|1  solicitud|1

-- Operación deshecha
BEGIN; ... ROLLBACK;
-- Resultado: borrador_rollback|0  items_rollback|0  solicitud_rollback|0

-- C1: catálogo sin servidor
SELECT * FROM vw_catalogo_disponible;
3|Coca 1.5L|80000|100
5|Limonada Feria|60000|30
1|Muzzarella|105000|40
2|Napolitana|150000|35
4|Pizza Feria|120000|15

-- C7: cantidad 0 y producto inexistente
CHECK constraint failed: cantidad > 0
FOREIGN KEY constraint failed
items_invalidos_guardados|0
```

C1 no referencia PostgreSQL: lee la foto local. C7 demuestra que el CHECK y la FK impiden dejar filas parciales.

2. Sincronizar y resolver

2.a La instantánea

La vista se define en PostgreSQL y se exporta a CSV. El precio viaja en centavos y la fecha se fija a la zona de negocio para que no dependa de la sesión que exporta.

```
CREATE OR REPLACE VIEW v_snapshot_movil AS
SELECT p.id AS producto_id, p.nombre,
       (p.precio * 100)::bigint AS precio_centavos,
       p.stock,
       (p.activo AND p.stock > 0) AS disponible,
       (NOT p.activo) AS eliminado,
       (now() AT TIME ZONE 'America/Argentina/Buenos_Aires')::date AS fecha_copia
FROM producto p;

\copy (SELECT producto_id, nombre, precio_centavos, stock, disponible,
              eliminado, fecha_copia
       FROM v_snapshot_movil ORDER BY producto_id)
  TO 'servidor_snapshot.csv' CSV HEADER
```

```
producto_id,nombre,precio_centavos,stock,disponible,eliminado,fecha_copia
1,Muzzarella,105000,40,t,f,2026-11-10
2,Napolitana,150000,35,t,f,2026-11-10
3,Coca 1.5L,80000,100,t,f,2026-11-10
4,Pizza Feria,120000,15,t,f,2026-11-10
5,Limonada Feria,60000,30,t,f,2026-11-10
```

En SQLite, .import --csv --skip 1 carga snapshot_servidor. Luego carga_snapshot.sql normaliza los booleanos t/f a 1/0 y actualiza producto_cache con ON CONFLICT (id) DO UPDATE. Resultado: 5 productos en caché, 5 disponibles.

2.b El semáforo

Estado	Cuándo	Qué hace
RECHAZADO	Producto inexistente, dado de baja, no disponible o stock insuficiente.	Rojo. Tiene prioridad sobre el precio; explica el problema en motivo.
CONFLICTO	No hay rojo, pero cambió el precio visto.	Amarillo. Guarda precio visto y precio actual para revisión humana.
SIMULADO_APROBADO	Todos los ítems coinciden con la instantánea.	Verde. No es una venta: es una simulación.

El script evalúa solo cola_sync.estado = PENDIENTE. Así, repetir la simulación no cambia nada. Un borrador sin ítems se rechaza. El motivo se construye por ítem, no con un texto fijo.

2.c Los casos

Caso	Preparación	Resultado real
C2	Precio de Muzzarella 105000 -> 110000; nueva instantánea.	CONFLICTO: "visto 105000, ahora 110000". Al aceptar y crear otra solicitud: SIMULADO_APROBADO.
C3	Stock de Coca 1.5L a 2; borrador por 5.	RECHAZADO: "stock insuficiente para producto 3 (pedido 5, disponible 2)".
C4	Baja lógica del producto 2.	RECHAZADO: "producto 2 dado de baja". La fila queda en caché con eliminado=1; el catálogo no la muestra.
C5	Borrador sin cambios.	SIMULADO_APROBADO: "Sin diferencias contra la instantánea".
C6	Reenvío del mismo UUID.	UNIQUE constraint failed: cola_sync.solicitud_uuid; el estado no cambia y el conteo sigue en 5.

```
-- C4: la fila queda, el catálogo la oculta
2|Napolitana|150000|0|1|2026-11-10
productos_2_en_catalogo|0
fila_2_en_cache|1

-- Control final en PostgreSQL
pedidos 5 | detalle_pedido 7 | stock_total 220
md5_pedidos  10c2d0044f769bd188ebc2ce804dded9
md5_detalles 2d6cd57a2046238bcd08f46e3f0c8723
pedidos_iguales: true | detalles_iguales: true | stock_igual: true
```

La tablet no creó pedidos ni movió stock. Los cambios de precio, stock y baja de C2-C4 fueron fixtures del servidor para preparar cada caso; el stock de C3 se restauró después de exportar su instantánea para que el control final vuelva al baseline.

3. Integrar y revisar

3.a Mapa de los tres motores

Motor	Qué guarda	Quién manda	Qué no guarda
PostgreSQL 18.6	Categorías, productos, precio, stock, pedidos, ventas, clientes, roles.	Fuente de verdad de precio, stock y venta.	Fotos, reseñas, borradores offline.
MongoDB (U7)	Ficha ampliada: imágenes, alérgenos, reseñas, atributos variables.	Contenido de la ficha; referencia producto_id.	Precio, stock, pedidos, datos personales.
SQLite 3.54	producto_cache, borradores, ítems, cola_sync.	No manda: solo propone y simula.	Clientes, ventas confirmadas, credenciales.

3.b Plan de mantenimiento

Pregunta	Respuesta
¿Qué se respalda y cómo se prueba?	PostgreSQL con pg_dump -Fc diario; SQLite con copia del archivo; CSV archivado. Se restaura en una base nueva, se comparan conteos y md5 y se verifica integridad.
Si se restaura un respaldo viejo de PostgreSQL...	La tablet no manda: se exporta una instantánea nueva, se recarga y el semáforo reevalúa. Un precio viejo da CONFLICTO; un producto que ya no está da RECHAZADO.
¿Cada cuánto y quién revisa CONFLICTO/RECHAZADO?	CONFLICTO: encargado del turno, al cierre. RECHAZADO: administrador de datos, diario. Cola sin resolver: revisión semanal.

3.c Code review con IA

#	Unidad	Hallazgo	Decisión
H-01	U2	carga_masiva.sql choca con el trigger de precio de TP2; el flujo documentado falla.	Aceptado
H-02	U3	indices.sql e indices_u3.sql duplican pedido(fecha) y detalle_pedido(producto_id).	Corregido
H-03	U3	La vista materializada de facturación filtra activo y borra ventas históricas.	Aceptado
H-04	U3/U4	date_trunc(month) depende de TimeZone de la sesión.	Aceptado
H-05	U3	Nuevo índice sobre pedido(cliente_id).	Descartado
H-06	U1	restricciones_tp2 no elimina el CHECK original de cantidad.	Aceptado

H-02 se corrigió en el laboratorio con correccion_indices.sql. H-05 se descartó porque idx_pedido_cliente ya existe. Las evidencias están en u8/resultados/code_review_pg.txt y correccion_indices.txt.

4. Kiro, bitácora y defensa

4.a Requisitos EARS

ID	Requisito	Caso
REQ-01	WHEN la tablet no tiene conexión THE SYSTEM SHALL mostrar el catálogo desde producto_cache.	C1
REQ-02	WHEN hay un precio distinto y ningún problema rojo THE SYSTEM SHALL marcar CONFLICTO.	C2
REQ-03	WHEN un producto no existe, está dado de baja, no disponible o sin stock THE SYSTEM SHALL marcar RECHAZADO.	C3/C4
REQ-04	WHEN todo coincide THE SYSTEM SHALL marcar SIMULADO_APROBADO; WHEN se repite el UUID SHALL rechazar el duplicado y no reevaluar.	C5/C6

4.b Bitácora de IA y DUIA

Entrada	Qué se pidió	Qué se aceptó / corrigió
1	Revisión de todos los scripts de la cursada.	Se aceptaron 5 hallazgos y se descartó el índice redundante.
2	Esquema SQLite y semáforo.	Se corrigió ON CONFLICT (faltaba WHERE true), la normalización t/f y el motivo por ítem.
3	Spec Kiro con cuatro requisitos EARS.	Se aceptó y se separó la idempotencia en REQ-04.
4	Vista de instantánea y exportación CSV.	Se aceptó precio en centavos y fecha de negocio; se descartó SELECT *.

La DUIA completa está en documentacion/u8/duia_u8.md. No se enviaron contraseñas, mails ni filas reales a la IA.

4.c README y defensa

README_U8.md contiene el orden para crear todo desde cero. La defensa de 10 minutos está en documentacion/u8/defensa_u8.md: mapa, un caso en vivo (C2 o C4), un hallazgo del code review y el plan de mantenimiento.

Cierre: el móvil propone, el servidor dispone. SIMULADO_APROBADO significa que los datos coinciden; no es una venta.

Anexo - Archivos y comandos

Archivo	Uso
sql/u8/schema_sqlite.sql	Cuatro tablas STRICT + snapshot + vista.
sql/u8/carga_snapshot.sql	Carga idempotente del CSV.
sql/u8/semaforo.sql	Clasificación de PENDIENTE.
sql/u8/pruebas_sqlite.sql	Borrador, ROLLBACK, C1 y C7.
sql/u8/caso_c2a.sql ... caso_c6.sql	Casos C2-C6.
sql/u8/pg_u8_lab.sql	Vista v_snapshot_movil y datos de feria.
u8/resultados/evidencia_u8.txt	Salida completa de la ejecución.
u8/servidor_snapshot.csv	Instantánea inicial.

```
# PostgreSQL de pruebas
# desde bases de datos/
psql -d foodstore_u8_lab -f sql/schema_completo.sql
psql -d foodstore_u8_lab -f sql/data.sql
psql -d foodstore_u8_lab -f sql/u8/pg_u8_lab.sql
psql -d foodstore_u8_lab -f sql/u8/exportar_snapshot.sql

# desde u8/
sqlite3 tablet_u8.db < ../sql/u8/schema_sqlite.sql
sqlite3 tablet_u8.db ".import --csv --skip 1 servidor_snapshot.csv snapshot_servidor"
sqlite3 tablet_u8.db < ../sql/u8/carga_snapshot.sql
sqlite3 tablet_u8.db < ../sql/u8/pruebas_sqlite.sql
sqlite3 tablet_u8.db < ../sql/u8/semaforo.sql
```